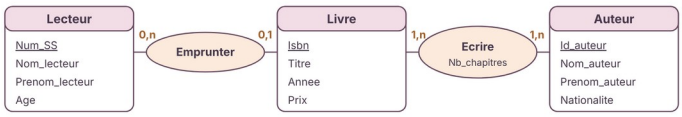
TERMINALE NSI · FICHE RÉSUMÉ

Bases de données et langage SQL

1 Le modèle entité-association

- **Entité** : objet identifiable (rectangle) ; son **identifiant** est souligné.
- **Association** : lien entre entités (ellipse), qui peut porter des attributs.
- **Cardinalité** (min, max), écrite côté entité : combien de fois l'entité participe.



Entités (rectangles, identifiant souligné) · associations (ellipses) · cardinalités (min, max)

Un lecteur emprunte 0 à n livres ; un livre est emprunté par 0 ou 1 lecteur.

2 Le modèle relationnel

Terme	Sens
Relation (table)	ensemble de lignes ayant les mêmes attributs
Attribut (colonne)	nom + domaine (valeurs autorisées)
Tuple (ligne)	une valeur par attribut
Clé primaire	identifie chaque ligne : unique, jamais NULL
Clé étrangère	fait référence à la clé primaire d'une autre relation

NOTATION D'UN SCHÉMA

```
Livre (Isbn, #Num_SS, Titre, Annee)
-- Isbn souligné = clé primaire
-- #Num_SS = clé étrangère vers Lecteur
```

Structure (les schémas) ≠ **contenu** (les lignes).

3 Du MEA aux relations

- Chaque **entité** → une **relation** ; identifiant → **clé primaire**.
- Association **(·,1)–(·,n)** : clé étrangère du côté du 1 (`Livre` reçoit `#Num_SS`).
- Association **(·,n)–(·,n)** : nouvelle relation, clé primaire = couple des clés étrangères (`Ecrire(#Isbn, #Id_auteur, Nb_chapitres)`).

> Au bac, le **modèle relationnel** est le plus souvent demandé ; un MEA peut être fourni pour être lu ou traduit.

4 Contraintes d'intégrité

Contrainte	Règle	Exemple de refus
de relation	clé primaire unique, non NULL	deux livres de même Isbn
de référence	une clé étrangère référence une clé existante	prêt à un lecteur inconnu ; supprimer un lecteur qui a un livre
de domaine	valeur dans le domaine	Num_SS = 'Paul'
utilisateur	règle métier	âge de 250 ans

LIRE UN CREATE TABLE

```
CREATE TABLE Livre (
  Isbn INTEGER, Num_SS INTEGER,
  Titre TEXT NOT NULL, Annee INTEGER,
  PRIMARY KEY (Isbn),
  FOREIGN KEY (Num_SS) REFERENCES Lecteur(Num_SS),
  CHECK (Annee BETWEEN -2000 AND 2100)
);
```

5 Redondance et anomalies

Une même information stockée plusieurs fois (le nom d'un auteur recopié sur chaque livre) provoque des anomalies :

- **mise à jour** : il faut modifier toutes les lignes, sans en oublier ;
- **insertion** : impossible d'ajouter un auteur sans livre ;
- **suppression** : supprimer le seul livre efface l'auteur.

> Remède : découper en plusieurs relations reliées par des clés étrangères.

6 Système de gestion de BDD

Service	Rôle
Persistance	les données survivent à l'arrêt de la machine
Accès concurrents	plusieurs utilisateurs en même temps, sans corruption
Efficacité	exécution optimisée des requêtes
Sécurisation	authentification, droits différents

Transaction : suite d'opérations en tout ou rien (virement). Propriétés **ACID** : atomicité, cohérence, isolation, durabilité.

Je sais…

- ☐ lire et traduire un MEA
- ☐ trouver clés primaires et étrangères
- ☐ dire quelle contrainte est violée
- ☐ repérer une redondance
- ☐ citer les services d'un SGBD
- ☐ écrire SELECT … WHERE … ORDER BY
- ☐ utiliser COUNT, SUM, AVG, MIN, MAX
- ☐ écrire une jointure (2 à 4 tables)
- ☐ écrire INSERT, UPDATE, DELETE
- ☐ lire une requête et la justifier

7 Interroger une table

```
SELECT Titre, Annee    -- colonnes (* = toutes)
FROM Livre             -- table
WHERE Annee > 1900     -- lignes gardées
ORDER BY Annee DESC;   -- tri (ASC par défaut)
```

Dans le WHERE	Sens
`= <> < <= > >=`	comparaisons
`AND OR NOT`	opérateurs booléens
`BETWEEN a AND b`	bornes incluses
`IN (…)`	dans une liste
`LIKE 'Le%'`	% = suite quelconque, _ = 1 caractère
`IS NULL`	valeur absente (jamais = NULL)

`DISTINCT` supprime les doublons ; `AS` renomme une colonne ; texte entre **apostrophes** : `'Camus'`.

8 Fonctions d'agrégation

Fonction	Renvoie
`COUNT(*)`	nombre de lignes
`COUNT(attr)`	nombre de valeurs non NULL
`SUM, AVG`	somme, moyenne (NULL ignorés)
`MIN, MAX`	plus petite, plus grande valeur

```
SELECT COUNT(*) FROM Livre WHERE Annee < 1900;
```

Résultat sur une seule ligne. ❌ pas d'agrégat dans un WHERE ; ❌ pas de colonne « normale » avec un agrégat (sans GROUP BY).

9 Jointures

```
SELECT Livre.Titre
FROM Livre
JOIN Ecrire ON Livre.Isbn = Ecrire.Isbn
JOIN Auteur ON Ecrire.Id_auteur = Auteur.Id_auteur
WHERE Auteur.Nom_auteur = 'Camus';
```

- `ON fk = pk` : on suit le chemin des clés étrangères ; n tables = n − 1 JOIN.
- Attribut présent dans deux tables : le **préfixer** (`Livre.Isbn`).
- Les lignes sans correspondance disparaissent ; penser à `DISTINCT`.

Pour aller plus loin : JOIN = INNER JOIN ; LEFT, RIGHT, FULL OUTER JOIN gardent aussi les lignes sans correspondance. **Au bac, seul JOIN est exigible** (éventuellement plusieurs).

10 Mettre à jour

```
INSERT INTO Lecteur (Num_SS, Nom_lecteur,
Prenom_lecteur, Age)
VALUES (206077512398765, 'Benali', 'Sofia', 20);
UPDATE Livre SET Num_SS = NULL WHERE Isbn =
9782070368228;
DELETE FROM Lecteur WHERE Nom_lecteur = 'Topor';
```

Toujours un WHERE : sans lui, UPDATE et DELETE touchent toute la table. Le SGBD refuse ce qui viole une contrainte.

+ Pour aller plus loin : GROUP BY

```
SELECT classe, COUNT(*) FROM Eleve
WHERE note >= 10        -- filtre des lignes
GROUP BY classe         -- une ligne par classe
HAVING COUNT(*) >= 2;   -- filtre des groupes
```

Ordre d'exécution : FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Dans le SELECT : attributs du GROUP BY et agrégats seulement.

11 Écrire une requête en 6 questions

Question	Clause
Que dois-je afficher ?	SELECT (colonnes, agrégat)
Dans quelles tables ?	FROM
Plusieurs tables ?	JOIN … ON fk = pk
Quelles lignes garder ?	WHERE
Un résultat « par » … ?	GROUP BY / HAVING (+ loin)
Doublons ? Ordre ?	DISTINCT, ORDER BY

Rédiger : phrases complètes, vocabulaire exact (« Num_SS est une clé étrangère qui fait référence à la clé primaire de Lecteur »).

! Les pièges classiques

- `= NULL` ne marche jamais : `IS NULL`.
- Texte entre apostrophes `'Paris'`, nombres sans.
- `COUNT(*)` ≠ `COUNT(col)` (NULL ignorés), `AVG` ignore les NULL.
- Colonne **ambiguë** : préfixer par la table.
- Ordre des clauses : SELECT, FROM, JOIN … ON, WHERE, (GROUP BY, HAVING), ORDER BY.
- `'Camus'` ≠ `'camus'` dans les données ; les mots-clés, eux, ne sont pas sensibles à la casse.